Infrastructure

Machine Learning Engineering

Lecturers: Tomáš Báča, Jan Brabec, Jan Lukáný

Lab Instructor: Ravi Kumar Thakur

Faculty of Electrical Engineering
Czech Technical University in Prague

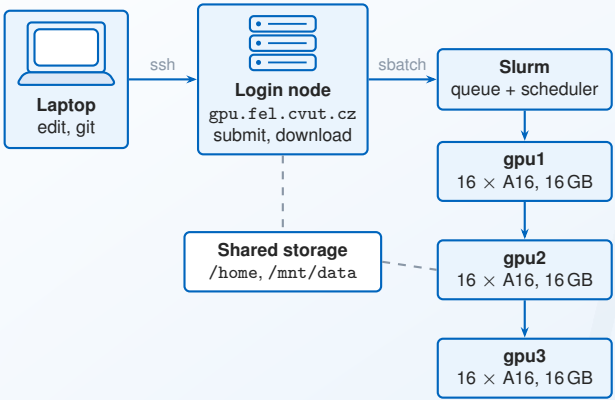
6 October 2026



FACULTY
OF ELECTRICAL
ENGINEERING
CTU IN PRAGUE



Nodes in the Cluster



Two kinds of machine

Login node: shared by everybody. Edit, `git pull`, download data, submit jobs.

Compute nodes: 48 GPUs. You get one only by **asking Slurm**.

Files in `/home` are the same on every node.

Note

Use login node fpr testing your code. For large model training, use compute node.

Slurm

Slurm manages the resource allocation on the GPU cluster. It puts you in a queue and starts your job when the resources are free.

Commands you will use today

`sinfo`	partitions and node states
`srun`	run now, interactively
`sbatch`	submit a script, run later
`squeue --me`	my pending and running jobs
`scancel <id>`	stop a job
`sacct -j <id>`	what a finished job used

Partitions = queues with a time limit

Partition	Max time
`fast` (default)	4 h
`1day`	1 day
`long`	3 days
`extralong`	21 days

Shorter jobs start sooner. Choose appropriate partition and time that fits your job.

Login Node

Interactive session on a GPU node

```
srun --gres=gpu:1 --cpus-per-task=4 --mem=8G \
     --time=00:30:00 --pty bash -i
hostname                          # gpu1, gpu2 or gpu3
nvidia-smi                        # your one A16
```

Verify with PyTorch

```
module load torchvision/0.22.1-foss-2024a-CUDA-12.6.0
python
>>> import torch
>>> torch.cuda.get_device_name(0)
'NVIDIA A16'
>>> exit()
```

Stay in this session: the next slides set up the lab in it.

What each flag asks for

`--gres=gpu:1`	one GPU
`--cpus-per-task=4`	CPU cores
`--mem=8G`	RAM (not GPU)
`--time`	then killed

Interactive vs batch

`srun --pty`: It is helpful in testing the code. For real training, use `sbatch`.

Set up the repository

1. Clone, and build the environment

```
git config --global user.name "Your Name"   # once
git config --global user.email you@fel.cvut.cz
git clone https://gitlab.fel.cvut.cz/thakurav/mle-
    practical.git
python -m venv --system-site-packages .venv
source env.sh                # module + venv
pip install -r requirements.txt
pytest                       # all tests pass
```

2. Get the data

```
dvc repro prepare            # downloads MNIST
git add dvc.lock && git commit -m "data: MNIST"
```

Module

PyTorch built on the cluster is accessed through **module**. The virtual enviornment adds the other dependencies like DVC, TensorBoard. The option `--system-site-packages` lets it see the module.

Why `prepare` runs first

It downloads MNIST once; every job then reads `data/mnist.pt`.

```python
device = torch.device("cuda")
model = make_model(width=512).to(device)  # weights
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)  # batch
        loss = loss_fn(model(x), y)        # forward
        opt.zero_grad()
        loss.backward()                    # gradients
        opt.step()                         # update
    torch.cuda.synchronize()  # wait, then time
```

from `src/train.py`

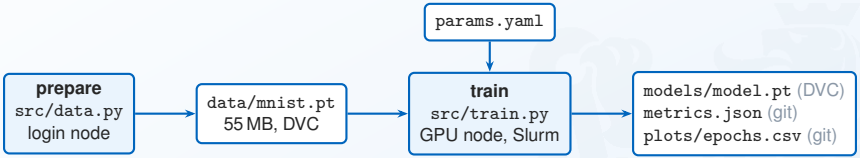
The only GPU-specific lines

- `.to(device)` for the **model** once
- `.to(device)` for **every batch**
- everything else is identical on CPU

Classic errors

- *Expected all tensors to be on the same device*: you forgot one `.to(device)`
- *CUDA out of memory*: batch or model too big for 16 GB
- timing without `synchronize()`: GPU calls return *before* the work is done

Training pipeline



Running one experiment with some params

```
dvc exp run -S width=1024     # not here: no GPU
dvc exp show                  # one row per run
```

`-S` overrides one value of `params.yaml` for this run only.

But `train` needs a GPU

So we never call `dvc exp run` on the login node. A Slurm job calls it for us: next slide.

Batch jobs: sbatch

slurm/train.sbatch

```
#!/bin/bash
#SBATCH --job-name=mnist
#SBATCH --partition=fast          # max 4 h
#SBATCH --gres=gpu:1              # one A16
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=00:20:00           # then killed
#SBATCH --output=logs/%x_%j.out   # name_jobid

source env.sh                     # module + venv
export TB_DIR="$SLURM_SUBMIT_DIR/runs"

nvidia-smi --query-gpu=name,memory.total --format=csv
dvc exp run --temp "$@"    # "$@" = your -S overrides
```

Submit and watch (login node)

```
sbatch slurm/train.sbatch     # prints the
     job id
squeue --me                   # PD / R
tail -f logs/mnist_<jobid>.out
sacct -j <jobid> \
  --format=JobID,State,Elapsed,MaxRSS
dvc exp show          # the finished run
```

Why batch jobs

You can log out and the job keeps running. The script is a **record** of exactly what ran, and it is in git.

`--temp`: each job runs in its own copy of the repo, so several jobs can run at once.

CPU vs GPU

Submit, changing one thing at a time

```
sbatch slurm/train.sbatch -S device=cpu
sbatch slurm/train.sbatch
sbatch slurm/train.sbatch -S batch_size=1024
sbatch slurm/train.sbatch -S width=4096
```

When all four are done

```
dvc exp show --drop 'Created|src/.*|data/.*'
```

Same params twice? DVC remembers the result and skips the training: the job ends in seconds.

Fill in

Device	Change	s/epoch	acc
cpu	—		
cuda	—		
cuda	batch 1024		
cuda	width 4096		

Let's Discuss

Why only a few times faster, not 100×? Which change helps the GPU most? What did the bigger batch cost?

Discord

https://discord.gg/s5XNpA3Yk



FACULTY
OF ELECTRICAL
ENGINEERING
CTU IN PRAGUE

